Dokumentacja funkcjonalna

System wizualizacji grafów planarnych

Projekt w języku C

Autorzy:
Krzysztof Wasilewski, Jakub Pietrzkiewicz

Data: 14.04.2026

Spis treści

1 Wstęp ... 3
1.1 Cel projektu ... 3
1.2 Dostępne funkcjonalności ... 3
2 Instrukcja użytkowania ... 4
2.1 Instalacja i kompilacja ... 4
2.2 Sposób uruchamiania ... 4
3 Parametry i opcje uruchamiania ... 5
3.1 Obowiązkowe argumenty ... 5
3.2 Opcjonalne argumenty ... 5
3.3 Przykłady wywołania ... 5
4 Format danych ... 6
4.1 Format pliku wejściowego ... 6
4.2 Format pliku wyjściowego tekstowego ... 6
4.3 Format pliku wyjściowego binarnego ... 6
5 Ograniczenia i wymagania ... 7
5.1 Wymagania systemowe i sprzętowe ... 7
5.2 Ograniczenia danych i struktury grafu ... 7
5.3 Specyfika algorytmów ... 7
6 Opis algorytmów ... 8
6.1 Algorytm Fruchterman-Reingold ... 8
6.2 Algorytm Tutte embeddings ... 9
6.2.1 Mechanizm automatycznego kotwiczenia ... 9
6.2.2 Matematyczny model wyznaczania współrzędnych ... 9
6.2.3 Proces iteracyjnej stabilizacji układu ... 10
6.2.4 Funkcjonalne właściwości rozwiązania ... 10
7 Obsługa błędów ... 11
7.1 Komunikaty o błędach ... 11
7.2 Zwracane kody powrotu programu ... 11
8 Przykłady użycia ... 12
8.1 Przykład 1: Prosty graf kwadratowy ... 12
8.2 Przykład 2: Graf z wagami i algorytm Tutte ... 12
8.3 Przykład 3: Eksport binarny z konfiguracją parametrów ... 12

1 Wstęp

1.1 Cel projektu

Celem projektu jest stworzenie aplikacji konsolowej w języku C, która umożliwia wizualizację grafów planarnych poprzez wyznaczanie optymalnych współrzędnych dla ich węzłów. Program przyjmuje na wejściu graf opisany w postaci listy krawędzi i generuje plik z współrzędnymi węzłów, które pozwalają na czytelną wizualizację struktury grafu.

Aplikacja implementuje dwa algorytmy układania grafów, co umożliwia porównanie ich efektywności i jakości generowanych wizualizacji. Program jest sterowany z linii poleceń za pomocą argumentów, co zapewnia elastyczność i możliwość automatyzacji.

1.2 Dostępne funkcjonalności

Program oferuje następujące funkcjonalności:

- Wczytywanie grafów planarnych z plików tekstowych w formacie listy krawędzi
- Wyznaczanie współrzędnych węzłów za pomocą wybranego algorytmu układania grafów
- Generowanie wyników w formacie tekstowym lub binarnym (do wyboru przez użytkownika)
- Obsługa dwóch algorytmów: Fruchterman-Reingold oraz Tutte Embedding
- Konfiguracja parametrów działania programu za pomocą argumentów linii poleceń

2 Instrukcja użytkowania

2.1 Instalacja i kompilacja

Program wymaga kompilatora C (np. GCC) oraz narzędzia make.

Kompilacja programu:

```
make
```

Czyszczenie plików kompilacji:

```
make clean
```

2.2 Sposób uruchamiania

Program jest uruchamiany z linii poleceń. Składnia wygląda następująco:

```
./graph [opcje] <plik_wejściowy> <plik_wyjściowy>
```

gdzie:

- `<plik_wejściowy>` - ścieżka do pliku z definicją grafu
- `<plik_wyjściowy>` - ścieżka do pliku wynikowego, gdzie program zapisze wynik działania

3 Parametry i opcje uruchamiania

3.1 Obowiązkowe argumenty

- `<plik_wejściowy>` - ścieżka do pliku tekstowego zawierającego definicję grafu w formacie listy krawędzi
- `<plik_wyjściowy>` - ścieżka do pliku, w którym zostaną zapisane wyniki (współrzędne węzłów)

3.2 Opcjonalne argumenty

- `-a <algorytm>` lub `--algorithm <algorytm>` - wybór algorytmu układania grafu
  - Dostępne wartości: `fruchterman`, `tutte`
  - Domyślnie: `fruchterman`
- `-f <format>` lub `--format <format>` - wybór formatu pliku wyjściowego
  - Dostępne wartości: `text`, `binary`
  - Domyślnie: `text`
- `-i <iteracje>` lub `--iterations <iteracje>` - liczba iteracji algorytmu
  - Wartość: liczba całkowita dodatnia
  - Domyślnie: `1000`
  - Dla algorytmu Fruchterman-Reingold: określa stałą liczbę kroków symulacji
  - Dla algorytmu Tutte: stanowi górny limit iteracji w przypadku braku wcześniejszej zbieżności
- `-t <temperatura>` lub `--temperature <temperatura>` - wartość początkowej temperatury dla algorytmu Fruchterman-Reingold
  - Wartość: liczba zmiennoprzecinkowa dodatnia
  - Domyślnie: `10.0`
- `-s <rozmiar>` lub `--size <rozmiar>` - rozmiar boku obszaru roboczego
  - Wartość: liczba zmiennoprzecinkowa dodatnia
  - Domyślnie: `1000.0`
- `-h` lub `--help` - wyświetla pomoc i dostępne opcje

3.3 Przykłady wywołania

Podstawowe wywołanie z domyślnymi parametrami:

```
./graph input.txt output.txt
```

Wybór algorytmu Tutte:

```
./graph -a tutte input.txt output.txt
```

Zapisanie wyniku w formacie binarnym:

```
./graph -f binary input.txt output.bin
```

Konfiguracja parametrów algorytmu Fruchterman-Reingold:

```
./graph -a fruchterman -i 2000 input.txt output.txt
```

Pełna konfiguracja parametrów:

```
./graph -a fruchterman -i 2000 -t 15.0 -s 2000 input.txt output.txt
```

4 Format danych

4.1 Format pliku wejściowego

Plik wejściowy jest plikiem tekstowym zawierającym listę krawędzi grafu. Każda linia opisuje jedną krawędź w następującym formacie:

```
<nazwa_krawędzi> <wierzchołek_A> <wierzchołek_B> <waga_krawędzi>
```

gdzie:
- `<nazwa_krawędzi>` - etykieta krawędzi (ciąg znaków bez spacji)
- `<wierzchołek_A>` - identyfikator pierwszego wierzchołka (liczba całkowita dodatnia)
- `<wierzchołek_B>` - identyfikator drugiego wierzchołka (liczba całkowita dodatnia)
- `<waga_krawędzi>` - waga krawędzi (liczba zmiennoprzecinkowa)

Poszczególne pola są oddzielone spacjami lub tabulatorami. Puste linie i linie z `#` są ignorowane.

Mechanizm przetwarzania: Program wykonuje dwa skany pliku. Pierwszy zlicza elementy i alokuje pamięć, drugi wczytuje dane.

Przykład pliku wejściowego:

```
# Przykładowy graf
AB   1  2  1.54
BC   2  3  1.0
CD   3  4  1.17
DA   4  1  1.93
AC   1  3  1.0
```

4.2 Format pliku wyjściowego tekstowego

Plik wyjściowy w formacie tekstowym zawiera listę współrzędnych węzłów. Każda linia opisuje pozycję jednego węzła:

```
<wierzchołek> <współrzędna_x> <współrzędna_y>
```

gdzie:
- `<wierzchołek>` - identyfikator wierzchołka (liczba całkowita)
- `<współrzędna_x>` - współrzędna X (liczba zmiennoprzecinkowa)
- `<współrzędna_y>` - współrzędna Y (liczba zmiennoprzecinkowa)

Przykład pliku wyjściowego:

```
1 0.0 0.0
2 1.0 0.0
3 1.0 1.0
4 0.0 1.0
```

4.3 Format pliku wyjściowego binarnego

Plik wyjściowy w formacie binarnym zawiera te same dane co format tekstowy, ale zapisane w reprezentacji binarnej dla efektywniejszego przechowywania i szybszego wczytywania. Dane zapisywane są jako bezpośredni zrzut z pamięci (Little-Endian dla architektury x86_64).

Każdy węzeł jest reprezentowany przez 20 bajtów, gdzie:
- 4 bajty: identyfikator wierzchołka (int)
- 8 bajtów: współrzędna X (double)
- 8 bajtów: współrzędna Y (double)

5 Ograniczenia i wymagania

5.1 Wymagania systemowe i sprzętowe

- **Architektura:** Wymagany procesor Little-Endian (np. x86_64).
- **Środowisko:** Kompilator GCC i narzędzie Make.
- **Pamięć:** Dla grafów mniejszych niż 5000 węzłów zużycie nie przekracza 1 GB RAM.

5.2 Ograniczenia danych i struktury grafu

- **Wierzchołki:** Identyfikatory muszą być dodatnimi liczbami całkowitymi.
- **Krawędzie:** Maksymalna długość etykiety to 32 znaki.
- **Struktura grafu:** Program obsługuje wyłącznie grafy spójne bez pętli własnych i bez multigrafów.

5.3 Specyfika algorytmów

- **Fruchterman-Reingold:** Złożoność $O((V^2 + E) \cdot I)$ - przy dużych grafach czas obliczeń rośnie znacząco.
- **Tutte:** Wymaga grafu planarnego i najlepiej 3-spójnego; inaczej możliwe nakładanie się wierzchołków.

6 Opis algorytmów

6.1 Algorytm Fruchterman-Reingold

Algorytm Fruchterman-Reingold jest algorytmem siłowym (force-directed) do wizualizacji grafów. Opiera się na modelu fizycznym, w którym wierzchołki oddziałują siłami, a krawędzie działają jak sprężyny. Celem jest układ w stanie równowagi o minimalnej energii.

Zasada działania

W modelu przyjmuje się następujące założenia:

- Wszystkie węzły w grafie odpychają się wzajemnie.
- Węzły połączone krawędzią przyciągają się do siebie.

Algorytm działa iteracyjnie i obejmuje następujące etapy:

1. Losowe rozmieszczenie węzłów w obszarze roboczym.
2. Obliczenie sił działających na każdy węzeł:
   - siły odpychającej od wszystkich pozostałych węzłów,
   - siły przyciągającej od węzłów połączonych krawędzią.
3. Przemieszczenie węzłów zgodnie z wypadkową sił.
4. Obniżenie temperatury ograniczającej maksymalne przemieszczenie węzłów.

Kroki 2-4 są powtarzane do momentu osiągnięcia stanu równowagi lub wykonania zadanej liczby iteracji.

Parametry algorytmu

- **Liczba iteracji** - określa czas działania algorytmu.
- **Temperatura początkowa** - wyznacza maksymalne przemieszczenie węzłów w początkowych iteracjach.
- **Waga krawędzi** - im większa waga, tym silniejsze przyciąganie między węzłami.

Opis matematyczny

a) Optymalny dystans

Optymalna odległość między węzłami dana jest wzorem:

$$k = \sqrt{\frac{\text{Obszar}}{|V|}}$$

gdzie:

- Obszar - całkowite pole powierzchni roboczej,
- $|V|$ - liczba węzłów w grafie.

b) Reguła odpychania

Siła odpychania dla dwóch węzłów oddalonych o odległość d:

$$f_r(d) = -\frac{k^2}{d}$$

c) Reguła przyciągania (z uwzględnieniem wag)

Siła przyciągania dla węzłów połączonych krawędzią o wadze w:

$$f_a(d) = w \cdot \frac{d^2}{k}$$

d) Wypadkowa sił

Dla każdego węzła obliczany jest wektor wypadkowy:

$$D_v = \sum f_r + \sum f_a$$

e) Stabilizacja układu (chłodzenie)

Aby zapobiec oscylacjom, maksymalne przesunięcie w pojedynczej iteracji ograniczone jest przez temperaturę t. Wartość ta maleje w kolejnych iteracjach aż do zera.

Zalety i wady

Zalety:
- generuje estetyczne i często symetryczne układy,
- zmniejsza liczbę przecięć krawędzi,
- ułatwia analizę struktury sieci dzięki grupowaniu silnie powiązanych węzłów,
- ma charakter uniwersalny.

Wady:
- wysoka złożoność obliczeniowa dla dużych grafów (każdy węzeł oddziałuje z każdym),
- możliwość zatrzymania w lokalnym minimum energii przy niekorzystnym doborze temperatury lub początkowego rozmieszczenia.

6.2 Algorytm Tutte embeddings

Algorytm Tutte'a wyznacza współrzędne wierzchołków grafu planarnego. Jego działanie opiera się na matematycznym modelu równowagi sił przyciągania, co pozwala na uzyskanie przejrzystej i uporządkowanej wizualizacji.

6.2.1 Mechanizm automatycznego kotwiczenia

Aby umożliwić rozpięcie grafu, program stosuje cztery statyczne punkty podparcia:

Identyfikacja wierzchołków bazowych: Wybierane są cztery wierzchołki o najwyższym stopniu. Przy remisie decyduje niższy ID. Wierzchołki sortowane jako V_1, V_2, V_3, V_4.

Definicja obszaru roboczego: Wierzchołki przypisane do narożników kwadratu o boku domyślnie 1000 jednostek („bok obszaru roboczego"):
- V_1: $(0, 0)$, V_2: $(\text{bok}, 0)$, V_3: (bok, bok), V_4: $(0, \text{bok})$

Statyczność ramy: Wierzchołki bazowe są wyłączone z iteracji, zmuszając resztę grafu do dopasowania.

Obsługa małych grafów: Dla mniejszych niż 4 węzłów stosowane jest automatyczne rozmieszczenie:
- **1 węzeł:** $\left(\frac{\text{bok}}{2}, \frac{\text{bok}}{2}\right)$
- **2 węzły:** $(0, 0)$ oraz (bok, bok)
- **3 węzły:** $(0, 0)$, $(\text{bok}, 0)$ oraz $\left(\frac{\text{bok}}{2}, \text{bok}\right)$

6.2.2 Matematyczny model wyznaczania współrzędnych

Pozycje wierzchołków wewnętrznych wyznaczane są przez rozwiązanie układu równań liniowych. Każdy wierzchołek i jest umieszczany w ważonym środku ciężkości swoich sąsiadów.

Współrzędne (x, y) każdego wolnego wierzchołka obliczane są według wzorów:

$$x_i = \frac{\sum_{j \in N(i)} w_{ij} \cdot x_j}{\sum_{j \in N(i)} w_{ij}}$$

$$y_i = \frac{\sum_{j \in N(i)} w_{ij} \cdot y_j}{\sum_{j \in N(i)} w_{ij}}$$

Legenda oznaczeń:
- x_i, y_i - wyznaczane współrzędne wierzchołka i
- $N(i)$ - zbiór wierzchołków sąsiadujących bezpośrednio z wierzchołkiem i
- w_{ij} - waga krawędzi łączącej wierzchołek i z wierzchołkiem j (pobrana z pliku wejściowego)

6.2.3 Proces iteracyjnej stabilizacji układu

Wyznaczenie współrzędnych nie jest operacją jednorazową, lecz procesem dążenia do równowagi Przebiega on w następujący sposób:

Inicjalizacja: Wierzchołki ramy trafiają do narożników kwadratu, a wszystkie pozostałe węzły są wstępnie umieszczane w centrum obszaru roboczego na pozycji $\left(\frac{\text{bok}}{2}, \frac{\text{bok}}{2}\right)$.

Iteracja: Program wielokrotnie przebiega przez listę wolnych wierzchołków, aktualizując ich pozycje na podstawie aktualnych położeń ich sąsiadów.

Warunek stopu: Algorytm Tutte kończy działanie, gdy spełniony zostanie jeden z dwóch warunków:
- maksymalne przesunięcie wierzchołka w danej iteracji spadnie poniżej zadanego progu precyzji $\varepsilon = 0.0001$ (układ osiągnął stabilność)
- liczba wykonanych iteracji osiągnie górny limit określony flagą `--iterations` (domyślnie 1000), co stanowi bezpiecznik w przypadku problemów ze zbieżnością.

6.2.4 Funkcjonalne właściwości rozwiązania

Zastosowanie powyższej metody gwarantuje następujące cechy:

Domknięcie wypukłe: Żaden wierzchołek ani krawędź nie wyjdzie poza ramę kwadratową.

Wypukłość ścian: Wewnętrzne obszary przedstawione jako wielokąty wypukłe.

Reprezentacja wag: Wyższe wagi dają mniejsze odległości między wierzchołkami.

Determinizm: Te same dane zawsze generują identyczny układ współrzędnych.

7 Obsługa błędów

7.1 Komunikaty o błędach

Program wyświetla komunikaty o błędach na standardowe wyjście błędów (stderr).

Główne kategorie błędów:

Błędy argumentów:
- `Error: Invalid number of arguments` - nieprawidłowa liczba argumentów
- `Error: Unknown option: <opcja>` - nieznana opcja
- `Error: Invalid algorithm name: <nazwa>` - nieprawidłowa nazwa algorytmu
- `Error: Invalid format: <format>` - nieprawidłowy format wyjściowy
- `Error: Invalid iteration count` - nieprawidłowa liczba iteracji (musi być > 0)
- `Error: Invalid temperature value` - nieprawidłowa wartość temperatury
- `Error: Invalid size value` - nieprawidłowa wartość rozmiaru obszaru (musi być > 0)

Błędy plików:
- `Error: Cannot open input file: <plik>` - nie można otworzyć pliku wejściowego
- `Error: Cannot create output file: <plik>` - nie można utworzyć pliku wyjściowego

Błędy danych:
- `Error: Invalid graph format at line <numer>` - nieprawidłowy format opisu grafu
- `Error: Duplicate edge at line <numer>` - duplikacja krawędzi
- `Error: Self-loop detected at line <numer>` - wykryto pętlę własną
- `Error: Graph is empty` - graf nie zawiera krawędzi

Błędy pamięci:
- `Error: Memory allocation failed` - brak pamięci

7.2 Zwracane kody powrotu programu

Program zwraca następujące kody wyjścia:

- `0` - sukces, program zakończył się prawidłowo
- `1` - błąd argumentów wiersza poleceń
- `2` - błąd otwarcia/utworzenia pliku
- `3` - błąd w formacie danych wejściowych
- `4` - błąd algorytmu (brak zbieżności, nieprawidłowe dane)
- `5` - błąd alokacji pamięci

8 Przykłady użycia

8.1 Przykład 1: Prosty graf kwadratowy

Plik wejściowy `square.txt`:

```
# Graf w kształcie kwadratu
AB  1  2  1.0
BC  2  3  1.0
CD  3  4  1.0
DA  4  1  1.0
```

Wywołanie programu:

```
./graph square.txt square_out.txt
```

Wynik: węzły ułożone w kwadrat, gotowe do wizualizacji.

8.2 Przykład 2: Graf z wagami i algorytm Tutte

Plik wejściowy `complex.txt`:

```
# Bardziej złożony graf
e1   1  2  1.0
e2   1  3  1.0
e3   2  3  1.0
e4   2  4  1.5
e5   3  4  1.2
e6   1  4  1.8
```

Wywołanie programu z algorytmem Tutte:

```
./graph -a tutte complex.txt complex_out.txt
```

Wynik: węzły ułożone zgodnie z algorytmem Tutte, gwarantujący brak przecięć dla grafu planarnego.

8.3 Przykład 3: Eksport binarny z konfiguracją parametrów

Plik wejściowy `large.txt`:

```
# Większy graf - 10 węzłów, 15 krawędzi
e1    1   2  1.0
e2    1   3  1.0
...
e14   3   4  1.5
e15   6   8  1.5
```

Wywołanie programu z pełną konfiguracją:

```
./graph --algorithm fruchterman --iterations 3000 \
               --temperature 20.0 --format binary \
               large.txt large_out.bin
```

Wynik: plik binarny `large_out.bin` zawierający współrzędne 10 węzłów, obliczone za pomocą algorytmu Fruchterman-Reingold z 3000 iteracjami i wyższą temperaturą początkową dla lepszej eksploracji przestrzeni rozwiązań.